

INDEX

1. INTRODUCTION
2. OBJECTIVES
3. FEATURES OF THE PROPOSED SYSTEM
4. INPUT AND OUTPUT OF PROPOSED SYSTEM
5. FRONT-END AND BACK-END 6.
- HARDWARE AND SOFTWARE 7.
- SCOPE AND LIMITATIONS
8. SOURCE CODE
9. OUTPUT
10. BIBLIOGRAPHY

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **MRS. SREEYA ACHARJEE** and **MISS ANKITA ROY**, my Computer Science teachers, for their valuable guidance, continuous encouragement, and support throughout the completion of this project.

I am also thankful to the **Principal, Mrs. MOUSUMI MAHAPATRA**, and school management for providing the necessary facilities and resources required for completing this project successfully.

Finally, I would like to thank my **teammates** who helped me in the successful completion of this project.

YASHIKA SUHASARIA

INTRODUCTION

The **Electricity Billing Management System** is a console-based application developed using **Python** as the front-end and **MySQL** as the back-end database. The main objective of this project is to computerize the electricity billing process and make it more efficient, accurate, and reliable.

This system helps in maintaining customer records, generating electricity bills, calculating charges based on electricity unit consumption, and storing billing history in a structured manner. It reduces manual effort, avoids calculation errors, and ensures quick retrieval of information.

The project demonstrates the practical use of Python programming along with database connectivity. It is designed mainly for educational purposes and can be

effectively used in small electricity offices for managing billing operations.

OBJECTIVES

- To design a menu-driven electricity billing system for a small electricity office.
- To implement a login feature for authorized access.
- To use MySQL for secure data storage and retrieval.
- To calculate bills based on a predefined slab structure.
- To apply late payment penalties automatically.
- To add, update, search, and delete customer records.
- To enable display and tracking of billing history.
- To reduce manual work and human errors

INPUT AND OUTPUT OF THE PROPOSED SYSTEM

- **Input:**

1. Login username and password
2. Customer ID
3. Customer name, mobile number, address
4. Previous and current meter readings
5. Month and year of billing
6. Due date

7. Number of days late

- **Output:**

1. Electricity bill details
2. Units consumed
3. Bill amount
4. GST amount
5. Late payment charges
6. Total payable amount
7. Confirmation messages for adding, updating, and deleting records

FEATURES OF THE PROPOSED SYSTEM

- The proposed system provides the following features:
 - User login authentication
 - Add new customer records
 - Update customer details
 - Delete customer records
 - Generate electricity bills
 - Automatic calculation of:
 - * Electricity charges
 - * GST

* Late fee

- View customer bills month-wise
- Secure storage of data using MySQL database
- Menu-driven and user-friendly interface

FRONT-END AND BACK-END

Front-end:

- Python (IDLE)
- Console-based interface for user interaction

Back-end:

- MySQL Database
- Tables used:
 - * Customer table
 - * Bill table

HARDWARE AND SOFTWARE

Hardware Requirements:

- Computer / Laptop
- Minimum 4 GB RAM
- Keyboard and Mouse
- Hard Disk

Software Requirements:

- Windows Operating System
- Python 3.x
- MySQL Server
- MySQL Connector for Python
- Python IDLE / Any Python IDE

SCOPE AND LIMITATIONS

Scope:

- Useful for small electricity offices
- Helps in digital record maintenance
- Reduces paperwork
- Fast bill generation
- Easy data retrieval
- Can be extended to:
 - Online payment system
 - Graphical User Interface (GUI)
 - Multi-user access

Limitations:

- Console-based (no GUI)
- Single-user system
- Manual data entry required
- Limited security
- Requires MySQL server to be running
- No online payment facility

SOURCE CODE

```
import mysql.connector as msc
import datetime

# ---- Connect to MySQL ----
con = msc.connect(
    host="localhost",
    user="root",
    password="2008",
    database="electricitydb"
)
cur = con.cursor()

# ---- Create Tables ----
cur.execute("""
CREATE TABLE IF NOT EXISTS customer(
    cid INT PRIMARY KEY,
    name VARCHAR(50),
    mob VARCHAR(10),
    addr VARCHAR(100),
    last_read INT DEFAULT 0
)
""")

cur.execute("""
CREATE TABLE IF NOT EXISTS bill(
    bid INT AUTO_INCREMENT PRIMARY KEY,
    cid INT,
```

```
mon INT,  
yr INT,  
units INT,  
amt FLOAT,  
gst FLOAT,  
late FLOAT,  
total FLOAT,  
due DATE,  
dayslate INT,  
FOREIGN KEY(cid) REFERENCES  
customer(cid) )  
""")
```

```
con.commit()
```

```
# ---- Bill Calculation ----
```

```
def calc(units, dayslate):
```

```
    if units <= 100:
```

```
        amt = units * 3
```

```
    elif units <= 200:
```

```
        amt = 100 * 3 + (units - 100) * 4
```

```
    else:
```

```
        amt = 100 * 3 + 100 * 4 + (units - 200) * 5
```

```
    gst = amt * 0.18
```

```
    late = dayslate * 10
```

```
    total = amt + gst + late
```

```
    return amt, gst, late, total
```

```
# ---- Generate Bill ----
```

```
def gen_bill():
```

```
cid = int(input("Enter Customer ID: "))
cur.execute("SELECT last_read FROM customer WHERE
cid={}".format(cid))
data = cur.fetchone()
```

```
if not data:
    print("Customer not found!")
    return
```

```
last = data[0]
print("Last Reading:", last)
curr = int(input("Enter Current Reading: "))
```

```
if curr < last:
    print("Current reading cannot be smaller!")
    return
```

```
units = curr - last
mon = int(input("Enter Month (1-12): "))
yr = int(input("Enter Year: "))
due = input("Enter Due Date (YYYY-MM-DD): ")
dl = int(input("Days Late: "))
```

```
amt, gst, late, total = calc(units, dl)
```

```
cur.execute(
                                                    "INSERT INTO
bill(cid,mon,yr,units,amt,gst,late,total,due,dayslate) "
    "VALUES({}, {}, {}, {}, {}, {}, {}, {}, '{}', {})".format(
        cid, mon, yr, units, amt, gst, late, total, due, dl
    )
```

```

    )
    cur.execute(
        "UPDATE customer SET last_read={} WHERE
cid={}".format(curr, cid)
    )

    con.commit()

    print("\nBill Generated")
    print("Units:", units)
    print("Amount: {:.2f}".format(amt))
    print("GST: {:.2f}".format(gst))
    print("Late Fee: {:.2f}".format(late))
    print("Total: {:.2f}\n".format(total))

# ---- View Bill ----
def view_bill():
    cid = int(input("Enter Customer ID: "))
    mon = int(input("Enter Month: "))
    yr = int(input("Enter Year: "))

    cur.execute(
        "SELECT * FROM bill WHERE cid={} AND mon={} AND
yr={}".format(cid, mon, yr)
    )

    data = cur.fetchall()

    if data:
        for r in data:

```

```
    print("\nBill ID:", r[0])
    print("Units:", r[4])
    print("Amount:", r[5])
    print("GST:", r[6])
    print("Late Fee:", r[7])
    print("Total:", r[8])
    print("Due Date:", r[9])
    print("Days Late:", r[10])
else:
    print("No bill found.")
```

---- Update Customer ----

```
def upd_cust():
    cid = int(input("Enter Customer ID: "))
    print("\n1. Name\n2. Mobile\n3. Address")
    ch = int(input("Enter choice: "))

    if ch == 1:
        val = input("New Name: ")
        cur.execute("UPDATE customer SET name='{}' WHERE
cid={}".format(val, cid))
    elif ch == 2:
        val = input("New Mobile: ")
        cur.execute("UPDATE customer SET mob='{}' WHERE
cid={}".format(val, cid))
    elif ch == 3:
        val = input("New Address: ")
        cur.execute("UPDATE customer SET addr='{}' WHERE
cid={}".format(val, cid))
    else:
```

```
    print("Invalid!")
    return
con.commit()
print("Customer updated")
```

---- Add Customer ----

```
def add_cust():
    cid = int(input("Enter Customer ID: "))
    name = input("Enter Name: ")
    mob = input("Enter Mobile: ")
    addr = input("Enter Address: ")
    read = int(input("Enter Initial Reading: "))

    cur.execute(
        "INSERT INTO customer VALUES({}, '{}', '{}', '{}', {})".format(
            cid, name, mob, addr, read
        )
    )

    con.commit()
    print("Customer added")
```

---- Delete Customer ----

```
def del_cust():
    cid = int(input("Enter Customer ID: "))
    cur.execute("DELETE FROM bill WHERE cid={}".format(cid))
    cur.execute("DELETE FROM customer WHERE
cid={}".format(cid))
    con.commit()
    print("Customer deleted")
```

---- Login ----

def login():

user = "admin"

pwd = "1234"

tries = 3

while tries > 0:

u = input("Username: ")

p = input("Password: ")

if u == user and p == pwd:

print("\n Login Successful\n")

return True

else:

tries -= 1

print("Wrong! Attempts left:", tries)

print("Access Denied")

return False

---- Main Menu ----

def main():

print("\n---- ELECTRICITY BILL SYSTEM ----")

while True:

print("""

1. Generate Bill

2. View Bill

3. Update Customer

4. Delete Customer

5. Add Customer

6. Exit

```
"""  
    ch = int(input("Enter choice: "))  
    if ch == 1:  
        gen_bill()  
    elif ch == 2:  
        view_bill()  
    elif ch == 3:  
        upd_cust()  
    elif ch == 4:  
        del_cust()  
    elif ch == 5:  
        add_cust()  
    elif ch == 6:  
        print("Goodbye !")  
        break  
    else:  
        print("Invalid choice!")  
  
# ---- Run Program ----  
if login():  
    main()  
  
con.close()
```